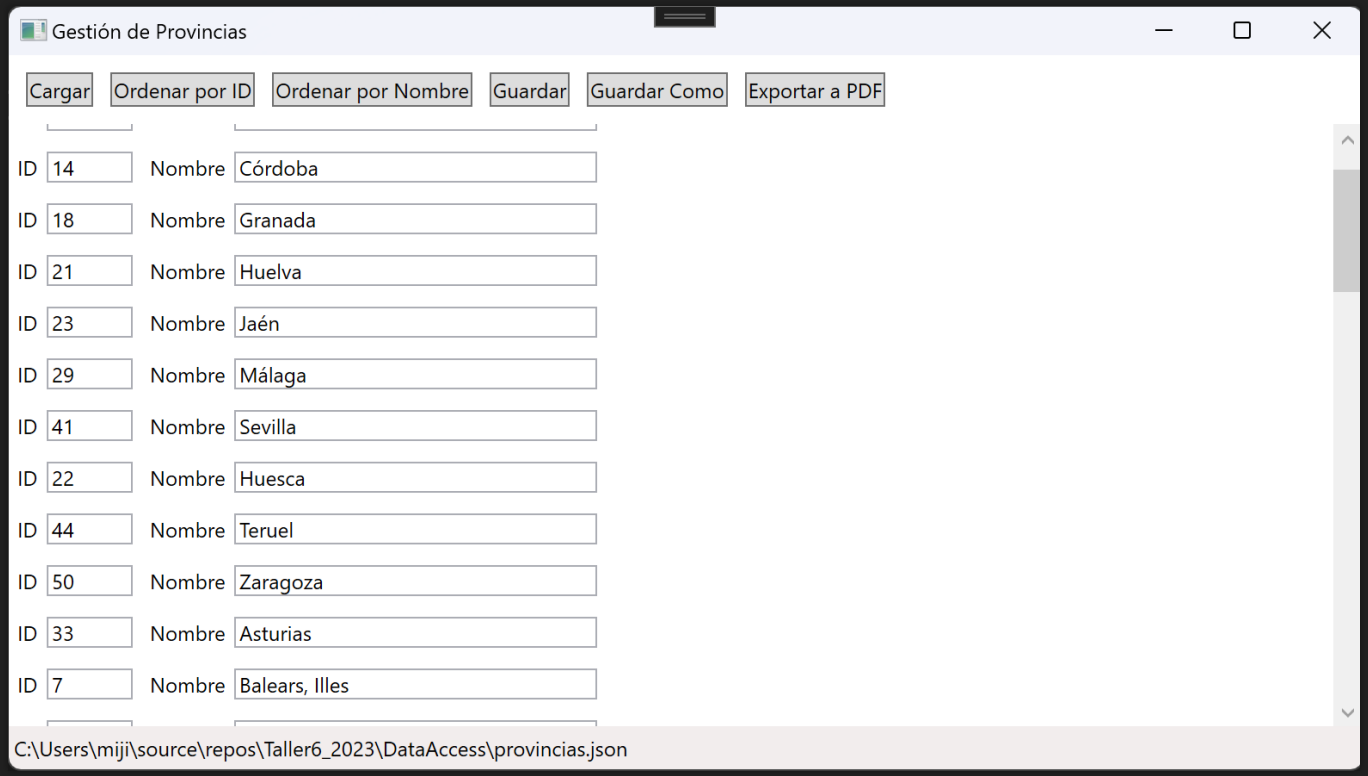
Acceso a datos JSON con WPF

Programación Avanzada en Entornos de Escritorio

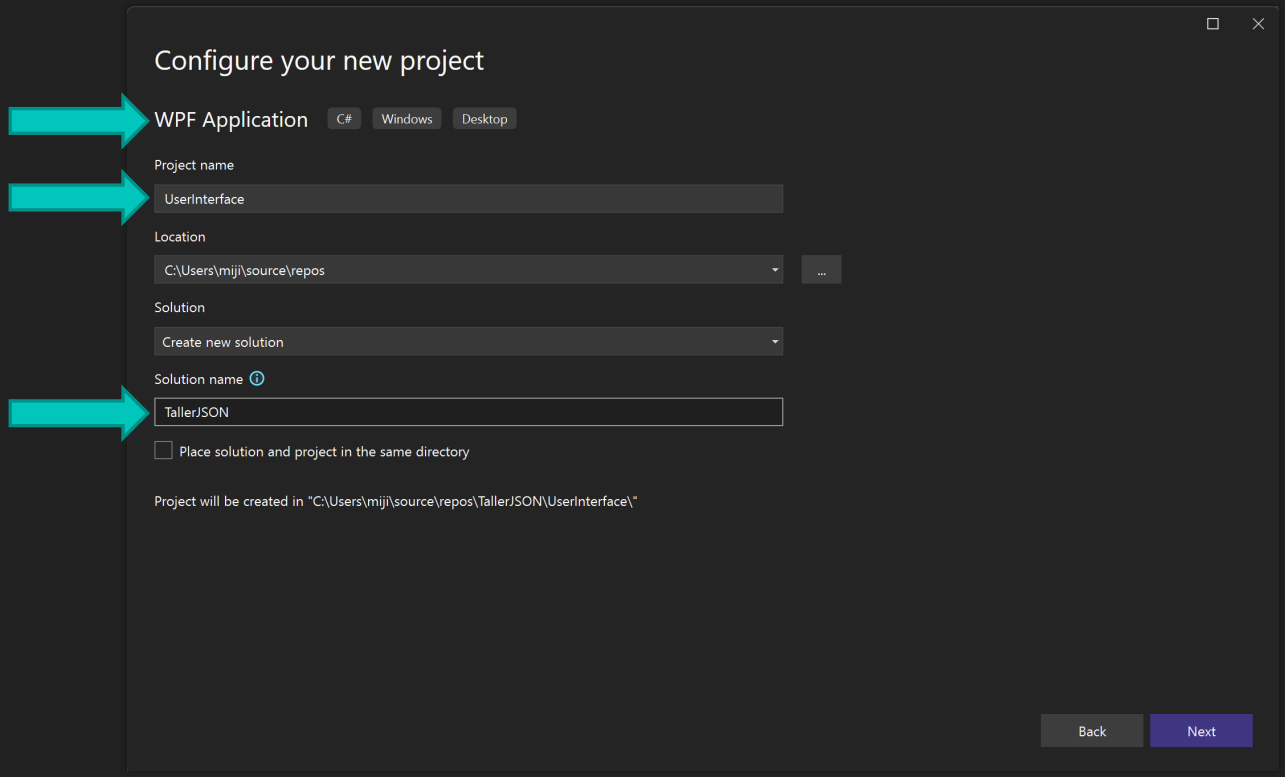
Miguel A. Teruel – materuel@dlsi.ua.es



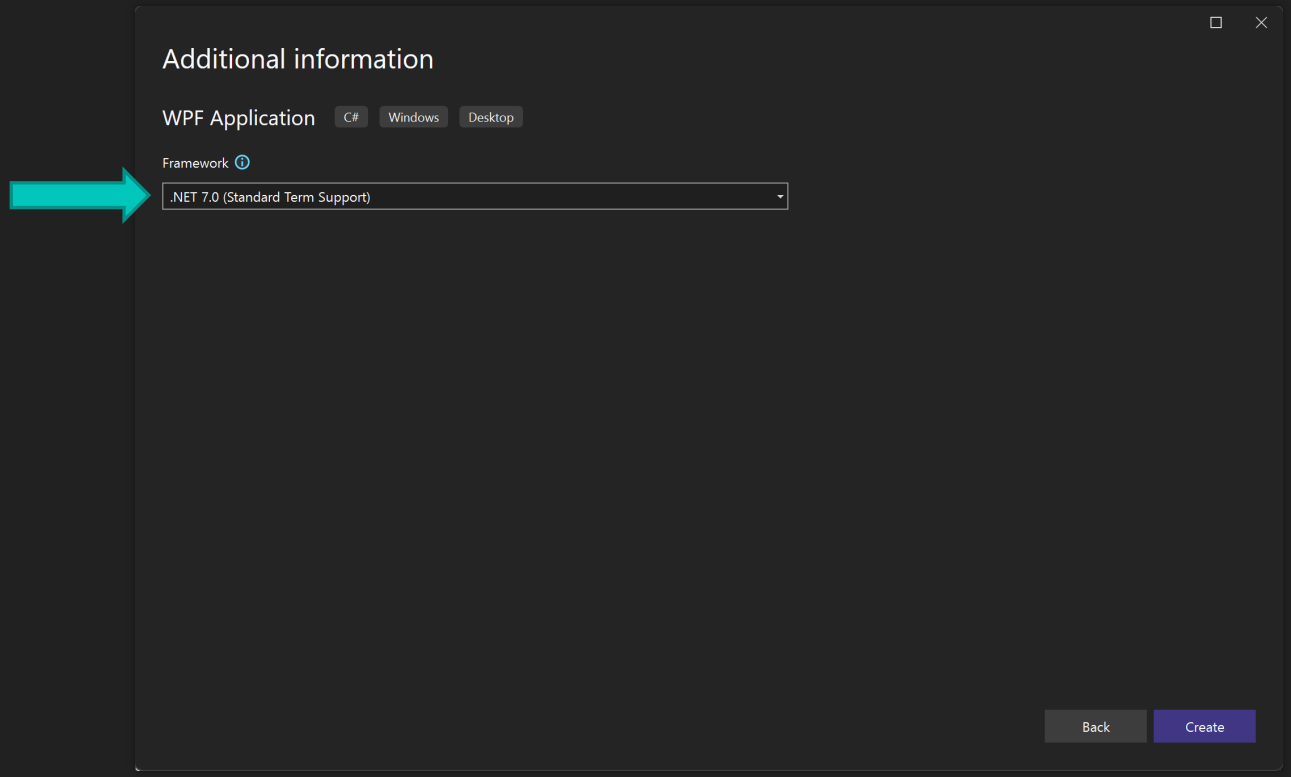
Objetivo


Gestión de Provincias
Cargar
Ordenar por ID
Ordenar por Nombre
Guardar
Guardar Como
Exportar a PDF
ID 14 Nombre Córdoba
ID 18 Nombre Granada
ID 21 Nombre Huelva
ID 23 Nombre Jaén
ID 29 Nombre Málaga
ID 41 Nombre Sevilla
ID 22 Nombre Huesca
ID 44 Nombre Teruel
ID 50 Nombre Zaragoza
ID 33 Nombre Asturias
ID 7 Nombre Balears, Illes
C:\Users\miji\source\repos\Taller6_2023\DataAccess\provincias.json

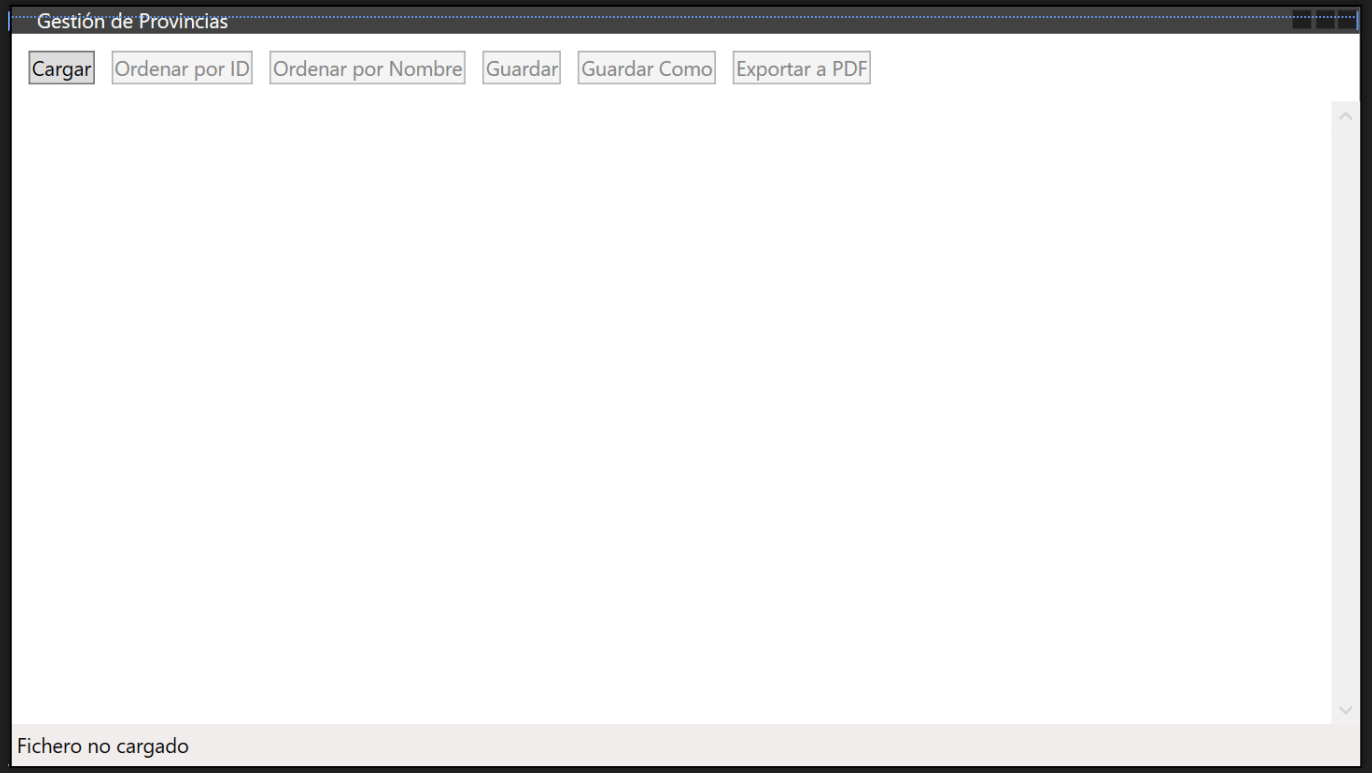
Creación de proyecto UserInterface



Creación de proyecto UserInterface



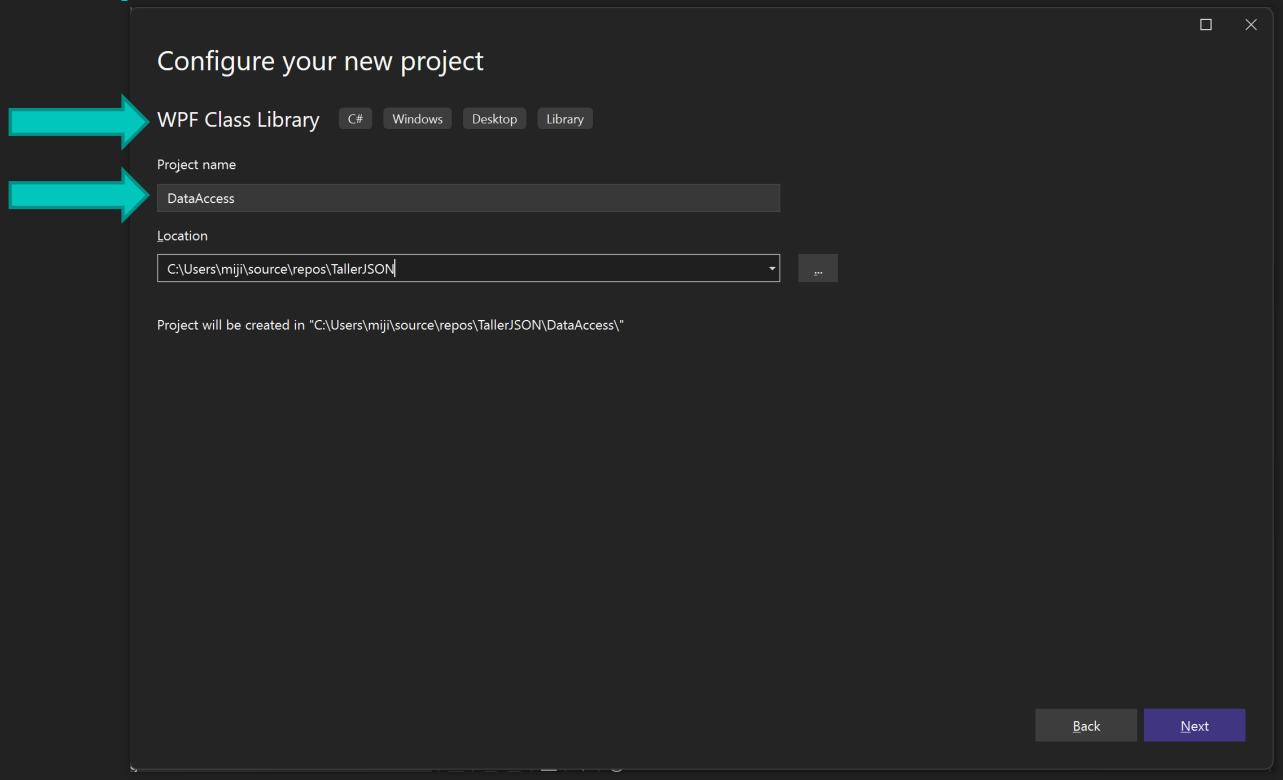
MainWindow.xaml



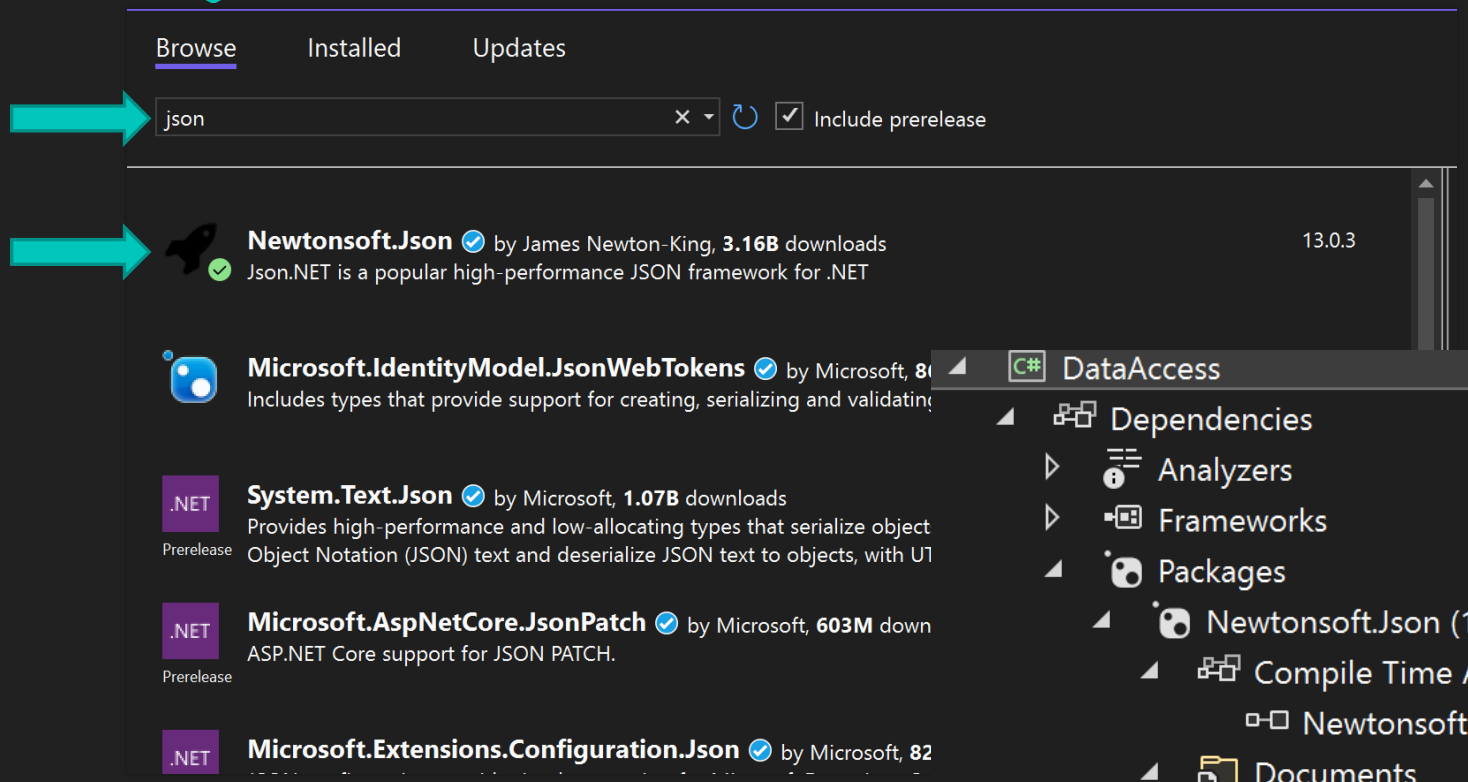
MainWindow.xaml

```xaml
<Window x:Class="UserInterface.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:UserInterface"
        mc:Ignorable="d"
        Title="Gestión de Provincias" Height="450" Width="800">
    <Grid>
        <StackPanel x:Name="botonesStackPanel" Height="40" Orientation="Horizontal" VerticalAlignment="Top">
            <Button x:Name="cargarButton" Content="Cargar" Margin="10,10,10,10" Click="cargarButton_Click"/>
            <Button x:Name="idButton" Content="Ordenar por ID" Margin="0,10,10,10" Click="idButton_Click" IsEnabled="False"/>
            <Button x:Name="nombreButton" Content="Ordenar por Nombre" Margin="0,10,10,10" Click="nombreButton_Click" IsEnabled="False"/>
            <Button x:Name="guardarButton" Content="Guardar" Margin="0,10,10,10" Click="guardarButton_Click" IsEnabled="False"/>
            <Button x:Name="guardarComoButton" Content="Guardar Como" Margin="0,10,10,10" Click="guardarComoButton_Click" IsEnabled="False"/>
            <Button x:Name="exportarButton" Content="Exportar a PDF" Margin="0,10,10,10" Click="exportarButton_Click" IsEnabled="False"/>
        </StackPanel>
        <ScrollViewer Margin="0,40,0,25">
            <StackPanel x:Name="provinciasStackPanel">
            </StackPanel>
        </ScrollViewer>
        <StatusBar Height="25" VerticalAlignment="Bottom">
            <StatusBarItem>
                <TextBlock Name="statusText" Text="Fichero no cargado" />
            </StatusBarItem>
        </StatusBar>
    </Grid>
</Window>
```

Creación de proyecto DataAccess



Paquete NuGet Newtonsoft.Json



Fichero JSON


```
{
  "ID": 4,
  "Nombre": "Almería"
},
{
  "ID": 11,
  "Nombre": "Cádiz"
},
{
  "ID": 14,
  "Nombre": "Córdoba"
},
{
  "ID": 18,
  "Nombre": "Granada"
},
{
  "ID": 21,
  "Nombre": "Huelva"
},
{
  "ID": 23,
  "Nombre": "Jaén"
},
{
  "ID": 29,
  "Nombre": "Málaga"
},
{
  "ID": 41,
  "Nombre": "Sevilla"
},
{
  "ID": 22,
  "Nombre": "Huesca"
},
{
  "ID": 44,
```


bit.ly/452mm9Y

Provincia.cs

```
namespace DataAccess
{
    8 references
    public class Provincia
    {
        3 references
        public int ID { get; set; }
        3 references
        public string Nombre {get; set;}
    }
}
```

ProvinciasJSON.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using Newtonsoft.Json;

namespace DataAccess
{
    3 references
    public class ProvinciasJSON
    {
        4 references
        public string Path {get;set;}

        1 reference
        public ProvinciasJSON(string path)
        {
            Path = path;
        }

        1 reference
        public List<Provincia> getProvincias()...

        2 references
        public bool saveProvincias(List<Provincia>? provincias)...
    }
}
```

ProvinciasJSON.cs

```
public List<Provincia> getProvincias()
{
    string jsonString = File.ReadAllText(Path);
    try
    {
        List<Provincia> provincias = JsonConvert.DeserializeObject<List<Provincia>>(jsonString);
        return provincias;
    }
    catch (JsonReaderException)
    {
        return null;
    }
}
```

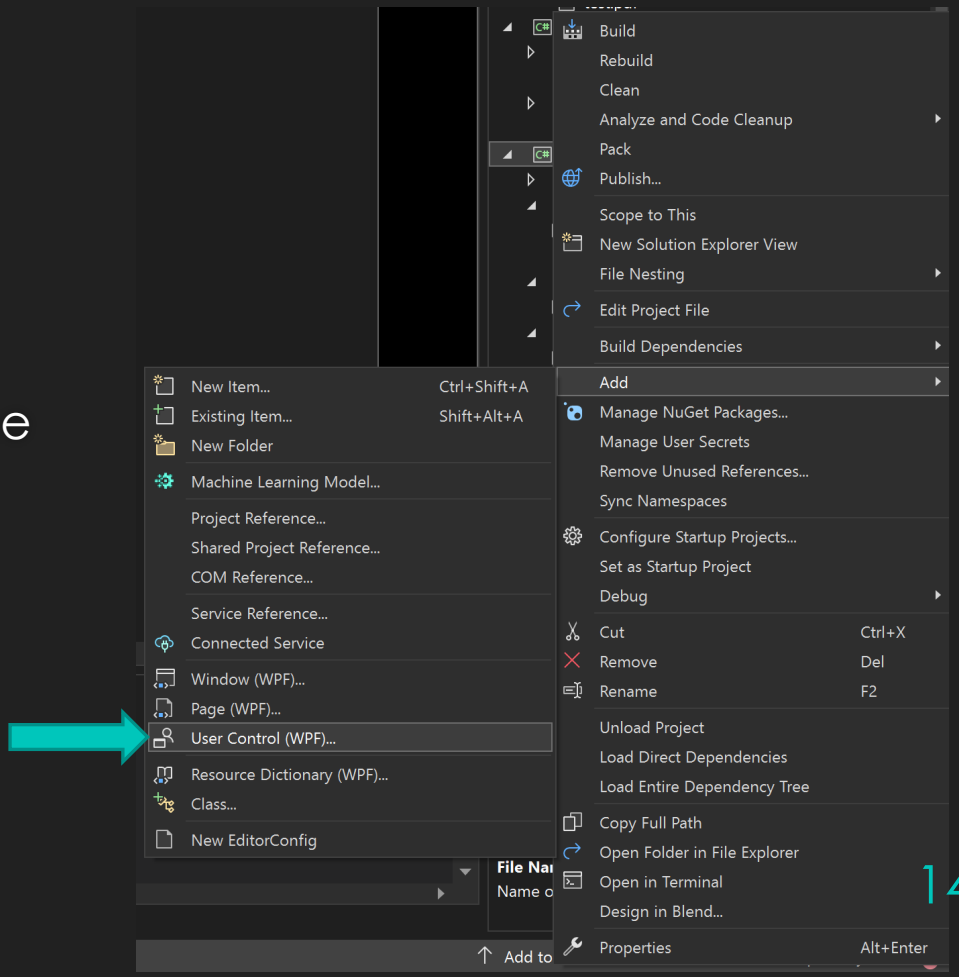
ProvinciasJSON.cs

```
public bool saveProvincias(List<Provincia> provincias)
{
    try
    {
        string jsonString = JsonConvert.SerializeObject(provincias);
        File.WriteAllText(Path, jsonString);
        return true;
    }
    catch (Exception)
    {
        return false;
    }

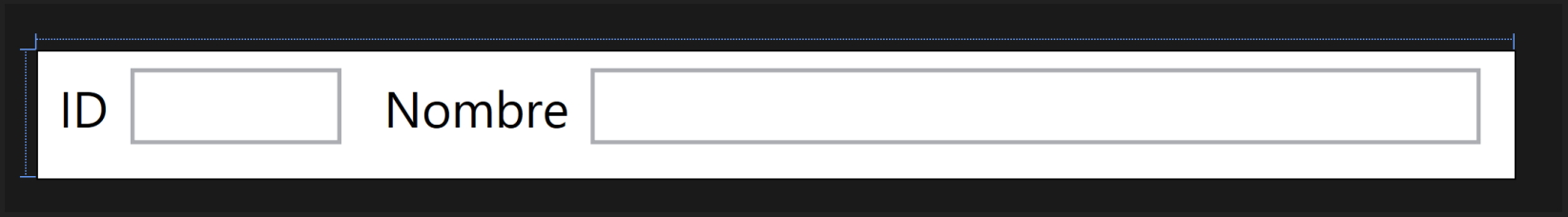
}
```

ProvinciaUserControl.xaml

- En proyecto UserInterface



ProvinciaUserControl.xaml




```
<UserControl x:Class="UserInterface.ProvinciaUserControl"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
             xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
             xmlns:local="clr-namespace:UserInterface"
             mc:Ignorable="d" d:DesignWidth="350" Height="30">
    <Grid>
        <Label Content="ID" HorizontalAlignment="Left" VerticalAlignment="Top"/>
        <TextBox x:Name="idTextBox" HorizontalAlignment="Left" Margin="22,4,0,0" TextWrapping="Wrap"
                 Text="" VerticalAlignment="Top" Width="50" LostFocus="idTextBox_LostFocus"/>
        <Label Content="Nombre" HorizontalAlignment="Left" VerticalAlignment="Top" Margin="77,0,0,0"/>
        <TextBox x:Name="nombreTextBox" HorizontalAlignment="Left" Margin="131,4,0,0" TextWrapping="Wrap"
                 Text="" VerticalAlignment="Top" Width="211" LostFocus="nombreTextBox_LostFocus"/>
    </Grid>
</UserControl>
```

ProvinciaUserControl.cs

```
using System;
using System.Windows;
using System.Windows.Controls;
using DataAccess;

namespace UserInterface
{
    /// <summary>
    /// Interaction logic for ProvinciaUserControl.xaml
    /// </summary>
    4 references
    public partial class ProvinciaUserControl : UserControl
    {
        private Provincia provincia;
        1 reference
        public Provincia Provincia...

        1 reference
        public ProvinciaUserControl()
        {
            InitializeComponent();
        }

        1 reference
        private void idTextBox_LostFocus(object sender, RoutedEventArgs e)...

        1 reference
        private void nombreTextBox_LostFocus(object sender, RoutedEventArgs e)...


    }
}
```

ProvinciaUserControl.cs

```
private Provincia provincia;
1 reference
public Provincia Provincia
{
    set
    {
        this.provincia = value;
        this.idTextBox.Text = value.ID.ToString();
        this.nombreTextBox.Text = value.Nombre;
    }

    get
    {
        return this.provincia;
    }
}
```

ProvinciaUserControl.cs

```
private void idTextBox_LostFocus(object sender, RoutedEventArgs e)
{
    try
    {
        provincia.ID = Convert.ToInt32(idTextBox.Text);
    }
    catch (FormatException)
    {
        provincia.ID = 0;
        idTextBox.Text = "0";
    }
}

1 reference
private void nombreTextBox_LostFocus(object sender, RoutedEventArgs e)
{
    provincia.Nombre = nombreTextBox.Text;
}
```

MainWindow.xaml.cs

```
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using DataAccess;
using ExportPDF;
using Microsoft.Win32;

namespace UserInterface
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    3 references
    public partial class MainWindow : Window
    {
        private ProvinciasJSON provinciasJSON;
        private List<Provincia> provincias;

        0 references
        public MainWindow()...

        2 references
        private void orderProvincias(bool nombre)...

        3 references
        private void showProvincias()...

        1 reference
        private void idButton_Click(object sender, RoutedEventArgs e)...

        1 reference
        private void nombreButton_Click(object sender, RoutedEventArgs e)...

        1 reference
        private void exportarButton_Click(object sender, RoutedEventArgs e)...

        2 references
        private void activarBotones(bool activar)...

        1 reference
        private void cargarButton_Click(object sender, RoutedEventArgs e)...

        2 references
        private bool guardar()...

        1 reference
        private void guardarButton_Click(object sender, RoutedEventArgs e)...

        1 reference
        private void guardarComoButton_Click(object sender, RoutedEventArgs e)...

        1 reference
        private void Window_Closing(object sender, System.ComponentModel.CancelEventArgs e)...
    }
}
```

MainWindow.xaml.cs

```
private void cargarButton_Click(object sender, RoutedEventArgs e)
{
    OpenFileDialog openFileDialog = new OpenFileDialog();
    openFileDialog.Filter = "Archivo JSON|*.json";
    if (openFileDialog.ShowDialog() == true)
    {
        provinciasJSON = new ProvinciasJSON(openFileDialog.FileName);
        provincias = provinciasJSON.getProvincias();
        if (provincias != null)
        {
            showProvincias();
            activarBotones(true);
            statusText.Text = openFileDialog.FileName;
        }
        else
        {
            provinciasStackPanel.Children.Clear();
            activarBotones(false);
            MessageBox.Show("Error cargando datos", "Error", MessageBoxButton.OK, MessageBoxImage.Error);
            statusText.Text = "Error cargando datos";
        }
    }
}
```

MainWindow.xaml.cs

```
private void showProvincias()
{
    provinciasStackPanel.Children.Clear();

    foreach (Provincia provincia in provincias)
    {
        ProvinciaUserControl provinciaUserControl = new ProvinciaUserControl
        {
            Provincia = provincia,
        };
        provinciasStackPanel.Children.Add(provinciaUserControl);
    }
}
2 references
private void activarBotones(bool activar)
{

    foreach (Button b in botonesStackPanel.Children)
    {
        if (b.Name != "cargarButton")
            b.IsEnabled = activar;
    }
}
```

MainWindow.xaml.cs

```
private bool guardar()
{
    bool res = provinciasJSON.saveProvincias(provincias);
    if (!res)
    {
        MessageBox.Show("Error guardando datos", "Error", MessageBoxButton.OK, MessageBoxImage.Error);
        return false;
    }
    else
    {
        MessageBox.Show("Datos guardados correctamente en " + provinciasJSON.Path, "Datos guardados", MessageBoxButton.OK, MessageBoxImage.Information);
        return true;
    }
}

1 reference
private void guardarButton_Click(object sender, RoutedEventArgs e)
{
    guardar();
}
```

MainWindow.xaml.cs

```
private void guardarComoButton_Click(object sender, RoutedEventArgs e)
{
    SaveFileDialog saveFileDialog = new SaveFileDialog();
    saveFileDialog.Filter = "Archivo JSON|*.json";
    if (saveFileDialog.ShowDialog() == true)
    {
        provinciasJSON.Path = saveFileDialog.FileName;
        if (provinciasJSON.saveProvincias(provincias))
        {
            statusText.Text = saveFileDialog.FileName;
            MessageBox.Show("Datos guardados correctamente en " + saveFileDialog.FileName, "Datos guardados", MessageBoxButton.OK, MessageBoxImage.Information);
        }
        else
        {
            MessageBox.Show("Error guardando datos en " + saveFileDialog.FileName, "Error", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

MainWindow.xaml.cs

```
private void Window_Closing(object sender, System.ComponentModel.CancelEventArgs e)
{
    if (provincias == null)
    {
        return;
    }

    switch (MessageBox.Show("¿Quiere guardar los datos antes de salir?", "Saliendo de la aplicación", MessageBoxButton.YesNoCancel, MessageBoxImage.Question))
    {
        case MessageBoxResult.Yes:
            if (guardar())
            {
                e.Cancel = false;
            }
            else
            {
                e.Cancel = true;
            }

            break;
        case MessageBoxResult.No:
            e.Cancel = false;
            break;
        case MessageBoxResult.Cancel:
            e.Cancel = true;
            break;

    }
}
```

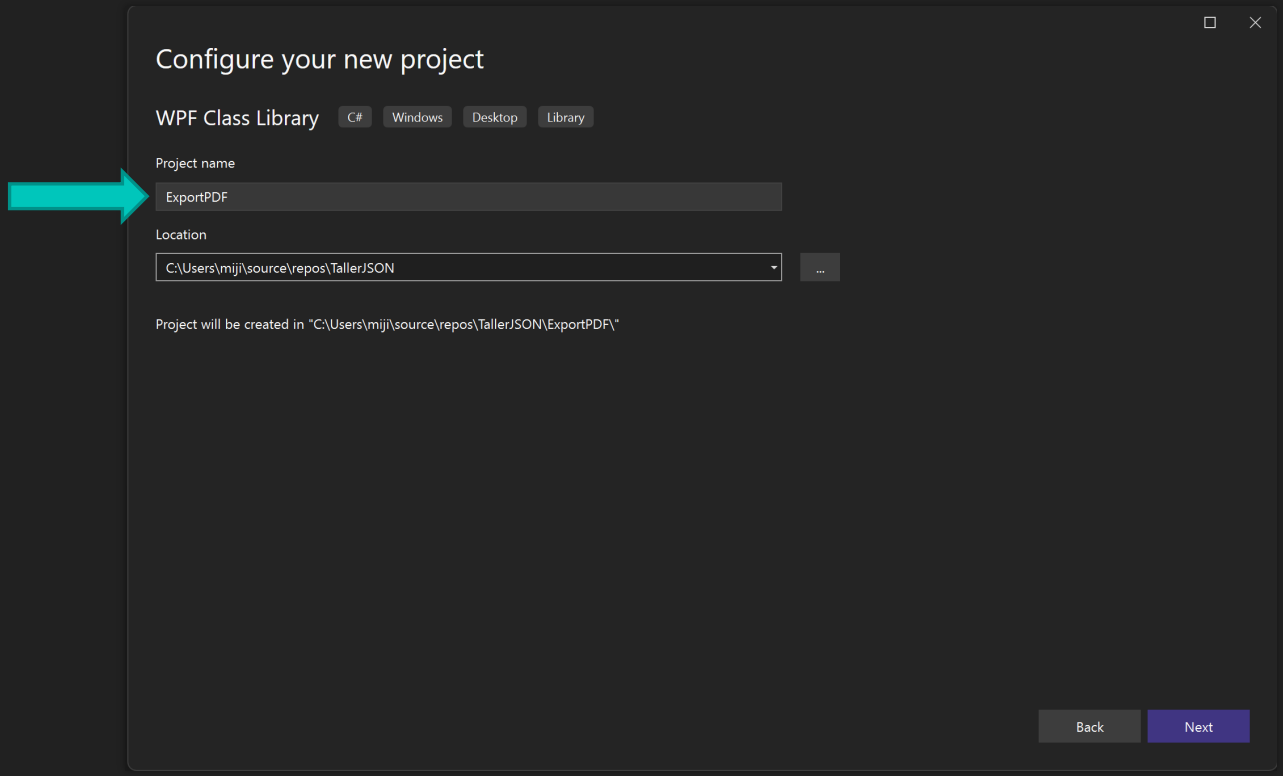
MainWindow.xaml.cs

```
private void orderProvincias(bool nombre)
{
    if (nombre)
        provincias = provincias.OrderBy(o => o.Nombre).ToList();
    else
        provincias = provincias.OrderBy(o => o.ID).ToList();
}

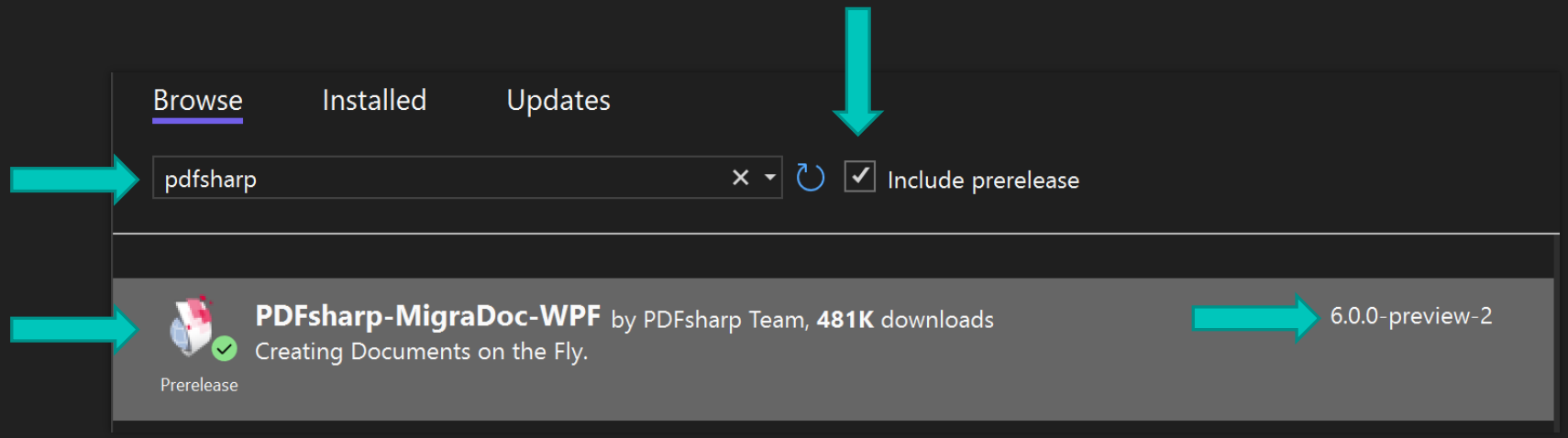
1 reference
private void idButton_Click(object sender, RoutedEventArgs e)
{
    orderProvincias(false);
    showProvincias();
}

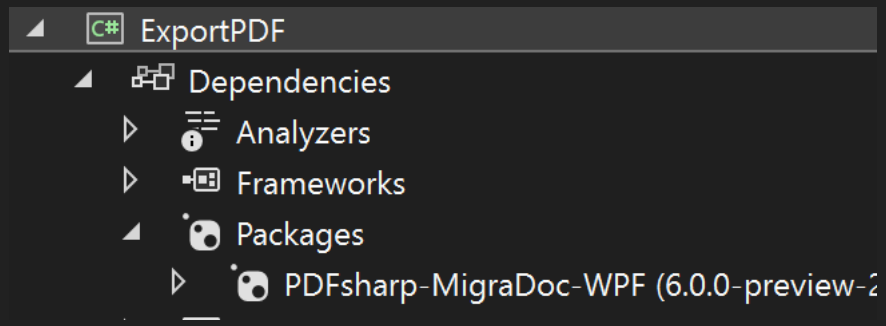
1 reference
private void nombreButton_Click(object sender, RoutedEventArgs e)
{
    orderProvincias(true);
    showProvincias();
}
```

Creación de proyecto ExportPDF



Paquete NuGet PDFsharp-MigraDoc-WPF





ProvinciasPDF.cs

```
using System;
using System.Collections.Generic;
using DataAccess;
using MigraDoc.DocumentObjectModel;
using MigraDoc.DocumentObjectModel.Tables;
using MigraDoc.Rendering;
using PdfSharp.Fonts;
using PdfSharp.Pdf;
using PdfSharp.Snippets.Font;

namespace ExportPDF
{
    2 references
    public class ProvinciasPDF
    {
        1 reference
        public bool SavePDF(List<Provincia> provincias, string path)...

    }
}
```

ProvinciasPDF.cs

```
try
{
    if (PdfSharp.Capabilities.Build.IsCoreBuild)
        GlobalFontSettings.FontResolver = new FailsafeFontResolver();

    Document document = new Document();
    Section section = document.AddSection();
    Paragraph paragraph = section.AddParagraph();
    paragraph.AddText("Provincias");
    paragraph.Format.Font.Size = 20;
    paragraph.Format.Font.Bold = true;
    paragraph.Format.Alignment = ParagraphAlignment.Center;

    section.AddParagraph();

    Table table = section.AddTable();
    table.Style = "Table";

    Column column = table.AddColumn("2cm");
    column.Format.Alignment = ParagraphAlignment.Center;
    column = table.AddColumn("10cm");
    column.Format.Alignment = ParagraphAlignment.Left;

    Row row = table.AddRow();
    row.HeadingFormat = true;

    row.Format.Font.Bold = true;
    row.Shading.Color = Colors.LightBlue;
    row.Cells[0].AddParagraph("ID").Format.Alignment = ParagraphAlignment.Center;
    row.Cells[1].AddParagraph("Provincia");

    foreach (Provincia p in provincias)
    {
        row = table.AddRow();
        row.Cells[0].AddParagraph(p.ID.ToString()).Format.Alignment = ParagraphAlignment.Center;
        row.Cells[1].AddParagraph(p.Nombre);
    }

    PdfDocumentRenderer pdfRenderer = new PdfDocumentRenderer
    {
        Document = document,
        PdfDocument = new PdfDocument()
    };

    pdfRenderer.RenderDocument();
    pdfRenderer.PdfDocument.Save(path);
    return true;
}
catch (Exception)
{
    return false;
}
```

MainWindow.xaml.cs

```
private void exportarButton_Click(object sender, RoutedEventArgs e)
{
    SaveFileDialog saveFileDialog = new SaveFileDialog();
    saveFileDialog.Filter = "Archivo PDF|*.pdf";
    if (saveFileDialog.ShowDialog() == true)
    {
        ProvinciasPDF provinciasPDF = new ProvinciasPDF();
        if (provinciasPDF.SavePDF(provincias, saveFileDialog.FileName))
        {
            if (MessageBox.Show("Datos exportados a PDF en " + saveFileDialog.FileName + ".\n\n ¿Desea abrirlo?",
                "Exportación correcta", MessageBoxButton.YesNo, MessageBoxImage.Information) == MessageBoxResult.Yes)
            {
                // Process.Start(saveFileDialog.FileName);
                var p = new Process();
                p.StartInfo = new ProcessStartInfo(saveFileDialog.FileName)
                {
                    UseShellExecute = true
                };
                p.Start();
            }
        }
    }
}
```

Ejercicios de ampliación

- Añadir más campos a las provincias (población, tamaño…)
- Añadir imágenes a las provincias (escudo, geografía…)
- Implementar paginación de provincias en lugar de scroll
- Añadir buscador
- Mejorar aspecto visual de la aplicación
- Mejorar exportación a PDF
- Exportar a más formatos (XML, CSV…)

Acceso a datos JSON con WPF

Programación Avanzada en Entornos de Escritorio

Miguel A. Teruel – materuel@dlsi.ua.es

